

Claude Cowork para el día a día

AI Generativa para Biomedicina · Universidad Austral

Autor: Paulo Veiga

Última Modificación: Junio, 2026



■ Agenda

1. De chat a agente: dónde encaja Cowork
2. Lo básico: interfaz, Instrucciones y Projects
3. Extender Cowork: Skills y el rol de los archivos MD / metadata
4. Conectar y automatizar: Connectors/MCP y Schedule
5. Compartir resultados: Live Artifacts
6. Advanced: Subagentes y Plugins

El hilo

Arrancamos ubicando a Cowork entre las tres superficies de Claude y explicando el cambio de paradigma de chat a agente (1). Después recorremos lo básico para empezar a trabajar: la interfaz, las Instrucciones y los Projects (2). Sobre esa base extendemos a Cowork con Skills y entendemos por qué los archivos Markdown / metadata son el lenguaje común del mundo LLM (3). Luego lo conectamos al mundo exterior con Connectors/MCP y lo volvemos proactivo con Schedule (4). Compartimos el resultado con Live Artifacts (5) y cerramos con dos piezas avanzadas, Subagentes y Plugins (6). El hilo conductor de toda la charla es una misión concreta — "Atlas", el analista de mercado que se arma pieza por pieza.

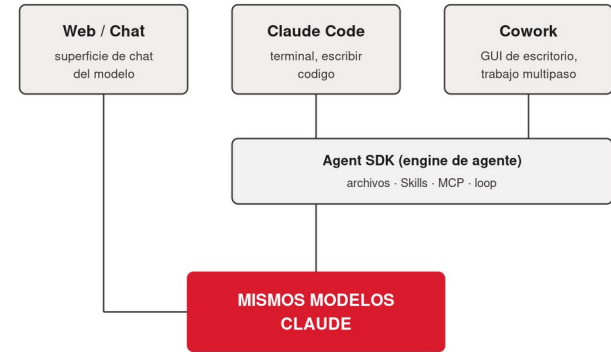
01

De chat a agente: dónde encaja Cowork

Que la audiencia entienda qué es Cowork frente a Claude Code y Web/Chat, e internalice el cambio de paradigma de "chatear" a "delegar un resultado" — el marco mental sobre el que se apoya toda la charla.

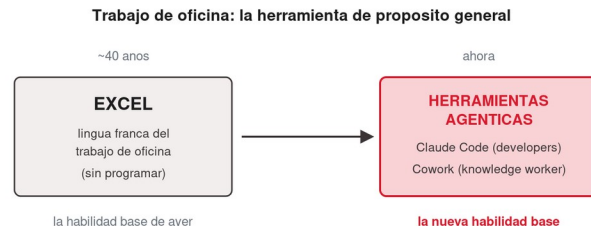
Las tres superficies de Claude

- Mismos modelos, distinta superficie: las tres caras — **Web/Chat**, **Claude Code** y **Cowork** — corren sobre **los mismos modelos Claude**. El matiz importa: **Cowork está construido sobre las mismas bases que Claude Code** (el **Claude Agent SDK**), así que Code y Cowork comparten el mismo engine de agente — los mismos archivos, las mismas Skills, el mismo MCP y el mismo loop de plan → aprobar → redirigir. **Web/Chat** es ese mismo modelo en una **superficie de chat**, no el loop agéntico completo.
- **Web/Chat** — navegador o app, sin instalar; chat, preguntas y tareas puntuales; público: todos.
- **Claude Code** — app de escritorio (pestaña Code + terminal); escribir, editar y publicar código; público: perfiles técnicos / developers.
- **Cowork** — app de escritorio (pestaña Cowork), GUI sin terminal; trabajo de varios pasos sobre archivos reales; público: knowledge workers sin terminal. **Esta charla vive acá.**



El superpoder de Cowork: la herramienta de propósito general del knowledge worker

- **La idea grande. Cowork es la herramienta de propósito general del knowledge worker** — de quien no programa. No un asistente de una tarea puntual: una herramienta horizontal para casi cualquier trabajo de conocimiento. Sin base técnica: el "lenguaje de programación" es el español.
- **La analogía que "pega" — "el nuevo Excel"** (encuadre de analistas / industria, no un claim de Anthropic). Así como Excel fue ~40 años la habilidad base del trabajo de oficina, las herramientas agénticas apuntan a ser **la nueva habilidad base**.
- **El encuadre oficial de Anthropic: Cowork como "Claude Code para el resto de tu trabajo"**.
- **Por qué te importa (bioingeniería).** La habilidad base del trabajo del conocimiento se redefine ahora; llegar temprano es ventaja concreta y portable.



De chat a agente: el cambio de paradigma

- La frase que resume toda la sesión: **"Dejás de tipear un mensaje a la vez y empezás a entregar un resultado."** El agente lo planifica, trabaja sobre tus archivos reales, y vos lo guiás — en lugar de hacer cada paso vos mismo.
- Cómo lo describe la propia Anthropic: trabajar con Cowork "se parece menos a una sesión de chat y más a asignarle tareas a un colega" ("less like a chat session and more like assigning tasks to a colleague"). Esa es exactamente la mudanza de paradigma de esta slide.
- Chatear vs delegar (no son dos productos, son dos formas de trabajar):

	Chatear	Delegar a un agente
Cómo trabajás	Un mensaje a la vez	Describís un resultado
Los pasos	Los hacés vos	El agente planifica y ejecuta
La salida	Texto en la ventana	Archivos en tu disco
Tu rol	Tipear el próximo prompt	Leer el plan, guiar a mitad de camino

ANTES (chat)



AHORA (agente / Cowork)



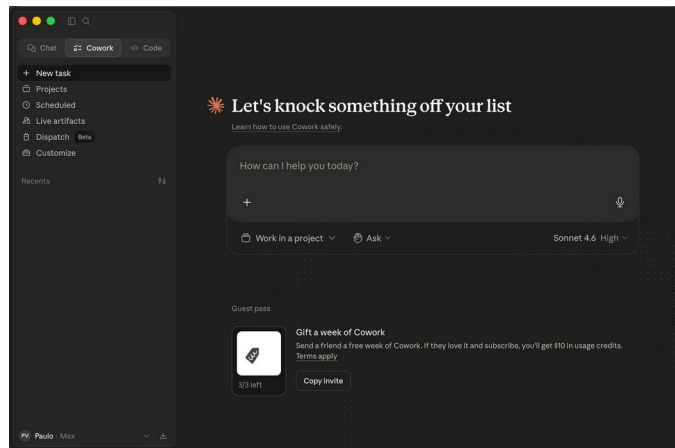
02

Lo básico: interfaz, Instrucciones y Projects

Dar a la audiencia lo mínimo para empezar a trabajar en Cowork: reconocer la interfaz, ajustar el comportamiento del agente con Instrucciones, y guardar todo de forma organizada con Projects.

(Demo time) Conozcamos la interfaz de Cowork

- **SLIDE DE DEMO EN VIVO** — Tour rápido de la pestaña Cowork sobre la app real, no sobre la slide.
- Anatomía de la pestaña Cowork (referencia anotada):
- Elementos a señalar en vivo: el selector de modo **"Ask"** (Ask before acting / Act without asking), el selector de carpeta de trabajo, la pestaña **Scheduled**, la pestaña **Live artifacts**, el panel de un **Project**.
- Control en Cowork = el dropdown de modo + los prompts de aprobar/redirigir por acción + el selector de carpeta. **No hay slash commands**: Cowork es GUI.



Los bloques de Cowork: cada problema, una pieza

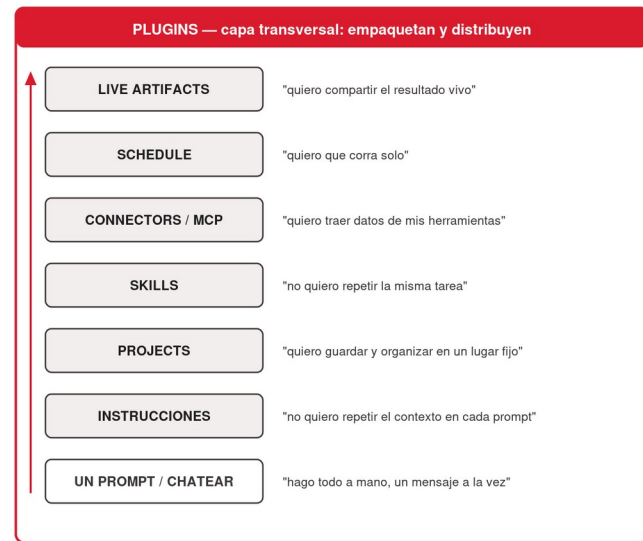
▪ **La idea.** Cowork no se aprende como una lista de features sueltas, sino como **bloques que se apilan**: cada bloque resuelve un problema concreto y recurrente, y al sumarse vuelven al agente cada vez más rico y autónomo. **Importante:** estos bloques **no** son una escalera estricta — usás solo los que tu tarea necesita; se combinan, no se exigen unos a otros.

▪ **Este es el mapa de toda la charla.** Vamos a **recorrer cada bloque, uno por uno**, en el resto de la sesión — en este orden. Volvé a esta slide como "estamos acá" cuando quieras ubicarte.

▪ **Cada bloque = un problema que ya tuviste:**

- **Un prompt / chatear** → hago todo a mano, un mensaje a la vez.
- **Instrucciones** → no quiero repetir el contexto en cada prompt.
- **Projects** → quiero guardar y organizar todo en un lugar fijo.
- **Skills** → no quiero repetir la misma tarea.
- **Connectors / MCP** → quiero traer datos de mis herramientas.
- **Schedule** → quiero que corra solo.
- **Live Artifacts** → quiero compartir el resultado vivo — un colaborador que entrega solo.

▪ **Plugins, transversal.** Hay una pieza que no es un bloque más en la pila: los **Plugins** son la **capa transversal de distribución** — empaquetan y reparten Skills, agentes y connectors a todos a la vez. La vemos al final de la charla (Sección 6).



Los bloques se apilan (cada uno suma autonomía); Plugins los distribuye a todos.

Instrucciones: ajustar el comportamiento sin repetirte

- **Concepto.** Las Instrucciones son el "contrato de trabajo" del agente: reglas en lenguaje natural que valen para todo lo que hagas, sin tener que repetirlas en cada prompt.
- **Ejemplo (Atlas) — qué podría decir un Instructions.** Quién es Atlas, qué empresas sigue, su audiencia, su tono y su regla de oro:
 - Una sola vez escribís esto; vale para todos los prompts del Project.
- **Cosas importantes a tener en cuenta.**
 - Mantenelas cortas y claras; son lenguaje natural, no código.
 - Sirven para evitar repetir lo mismo en cada prompt: lo que decís una vez vale para todo el Project.
 - Es el lugar para fijar reglas no negociables (como el disclaimer legal).

Sos Atlas, el analista de mercado de un equipo de trabajo.

Preparás un pulso semanal para colegas no técnicos (incluido el jefe), que se lee en 2 minutos antes de la reunión de los lunes.

- Empresas que seguís: Apple, Microsoft, Nvidia.
- Escribís en español, claro y breve, sin jerga financiera.
 - Si usás un término técnico, lo explicás en una línea.

· REGLA DE ORO: tus reportes son informativos y de uso interno.

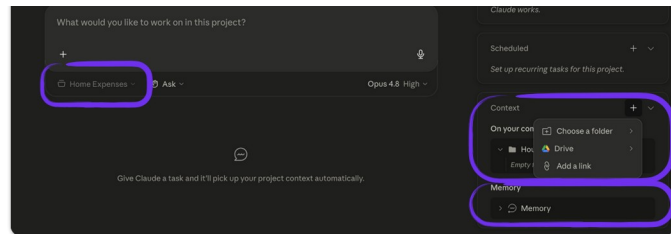
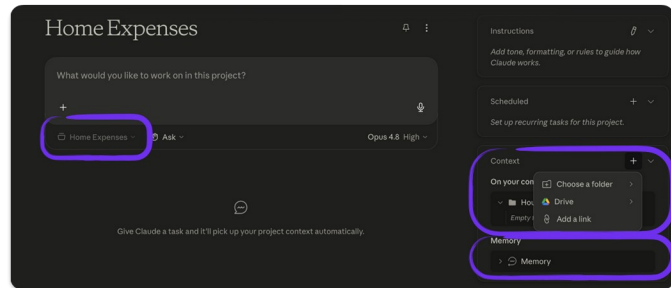
NUNCA son recomendaciones de inversión ni asesoramiento financiero.
Siempre incluí esa aclaración al final.

■ Projects: guardar todo en un lugar fijo

- **Concepto.** Un Project es un espacio de trabajo autocontenido: le da al agente una **carpeta propia**, **memoria** dentro del proyecto y un **lugar fijo** para sus tareas. Tiene tres capas persistentes: Instrucciones, base de conocimiento (Knowledge base) y Chats.
- **Ventajas.** Todo queda organizado y reutilizable: las Instrucciones valen para todo el Project, la memoria recuerda tus correcciones y preferencias, y los archivos viven en una carpeta concreta de tu disco.
- **Cosas importantes a tener en cuenta.**
 - Los chats dentro de un mismo Project **no comparten contexto entre sí** — solo se comparte la base de conocimiento.
 - En Cowork, qué carpetas se conceden lo controla el **selector de carpetas del sistema operativo**, no un archivo de configuración.
 - Buena práctica: una carpeta de trabajo dedicada, para saber siempre qué está en alcance (y nunca conceder una carpeta con datos confidenciales o credenciales).

El selector de carpetas y el panel de contexto

- **Conceder una carpeta de trabajo.** Cowork no toca tu disco por sí solo: vos le concedés una carpeta con el **selector de carpetas del sistema operativo** (el mismo folder picker que usás para abrir cualquier archivo). Lo que quede fuera de esa carpeta, Cowork no lo ve.
- **El panel de contexto del Project.** Es donde viven las Instrucciones, la base de conocimiento y la carpeta concedida — la "foto" de todo lo que el agente tiene a mano para ese Project.
- **Nota de seguridad.** El selector es a la vez tu garantía y tu límite: Cowork solo trabaja sobre lo que le concedés. **Nunca concedas una carpeta con datos sensibles, credenciales o información bajo NDA.** Buena práctica: una carpeta dedicada por Project, para saber siempre qué está en alcance.



03

Extender Cowork: Skills y el rol de los archivos MD / metadata

Que la audiencia sepa crear y habilitar Skills en Cowork (con su trampa específica del Save), y entienda — como sideway de alto nivel — por qué los archivos Markdown con metadata son el lenguaje común sobre el que se configura todo en el mundo LLM.

Skills: enseñarle a Claude algo una sola vez

- **Concepto.** Una Skill es una instrucción reutilizable (+ scripts opcionales) que el agente carga cuando tu pedido coincide con su descripción. Un trabajo por Skill: "si escribís 'y además', dividila en dos". La frase clave: "Todo lo que le explicás a Claude dos veces es una Skill que deberías escribir una vez."
- **Cómo se crea una Skill en Cowork.** Dos caminos reales (Cowork es GUI: **no hay slash commands**):
 - 1. **Pedísela en lenguaje natural** — "armame una Skill que haga X". Claude **escribe el archivo `SKILL.md`**, pero Cowork **NO la registra/habilita** solo. Tenés que **habilitarla** en **Customize** → **Skills** (el botón **Save skill / Save to enable**). Recién ahí queda activa.
 - 2. **Subís un ZIP** — empaquetás la carpeta de la Skill como `.zip` y la cargás en **Customize** → **Skills** → **"+"** → **Create skill** → **Upload a skill**, y la activás con el toggle.
- **Requisito.** Las Skills necesitan **Code execution** habilitado (Settings → Capabilities).
- **OJO — la trampa del Save (camino 1).** Es el error más común: pedís la Skill, Claude escribe el archivo... pero si no le das **Save / enable** en Customize, no queda habilitada y parece que "no funciona".
- **Ejemplo (Atlas).** La Skill `reporte-semanal`: lee TODOS los archivos crudos de una carpeta `fuentes/` (uno por portal), consolida por empresa y genera un reporte con formato fijo. La empresa más relevante va primera (★). Guarda con sufijo `new` para no pisar el ejemplo.

■ (Sideway) Archivos MD y metadata: el lenguaje común del mundo LLM

- **Concepto.** Casi todo lo que configurarás alrededor de un agente — Instrucciones, Skills (`SKILL.md`), archivos de agentes, docs de plugins, salidas — es texto plano en **Markdown** (`.md`). Markdown es la lingua franca del mundo LLM.
- **Qué es la metadata / los headers.** Muchos de esos archivos arrancan con un bloque de **metadata** (un "header" en YAML): por ejemplo, un `SKILL.md` declara `name` y `description`. Esa descripción es lo que dispara la Skill — de forma semántica, no por palabra clave.
- **Por qué esto es importante en el mundo LLM.**
 - El modelo lee texto: si la configuración es texto plano legible, el agente la entiende directamente, sin formato propietario.
 - La metadata le dice al sistema qué es cada archivo y cuándo usarlo (la `description` de una Skill decide cuándo se activa).
 - Es portable y versionable: el mismo estándar `SKILL.md` funciona entre herramientas (Cowork y Codex usan el mismo formato).
- **Nota de alcance:** esto es un sideway de alto nivel — qué es y por qué importa. No entramos en el detalle del formato de archivos.

Anatomía de un SKILL.md

- Así se ve un `SKILL.md` por dentro: un **bloque de metadata** arriba y el **cuerpo de instrucciones** abajo. Nada más — es texto plano.
- **La metadata (el header).** `name` identifica la Skill; `description` es lo que **decide cuándo se activa** — de forma semántica, no por palabra clave exacta. Una buena `description` = la Skill se dispara cuando corresponde.
- **El cuerpo.** Markdown común: los pasos que el agente sigue cuando la Skill se activa.
- Nota de alcance: reforzamos el sideway anterior (MD + metadata) con un ejemplo tangible — no entramos en el detalle fino del formato.

```
---
name: reporte-semanal
description: Genera el pulso semanal
a partir de la carpeta fuentes/.
Usar cuando pidan "reporte semanal".
---
```

METADATA (YAML)

"que es" +
"cuando se activa"

```
# Reporte semanal

1. Lee TODOS los archivos de
fuentes/ y consolida por empresa.

2. Genera el reporte con esta
estructura exacta...

3. Guarda con sufijo -new (no pises
el original).
```

CUERPO (Markdown)

"que hace":
las instrucciones

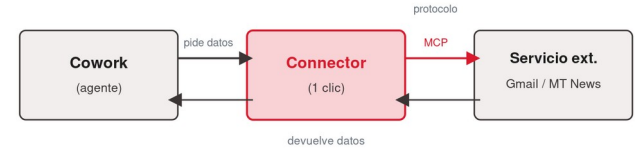
04

Conectar y automatizar: Connectors/MCP y Schedule

Que la audiencia entienda cómo Cowork toca el mundo exterior (Connectors vía MCP) y cómo pasa de reactivo a proactivo (Schedule), con la trampa de "la app tiene que estar abierta".

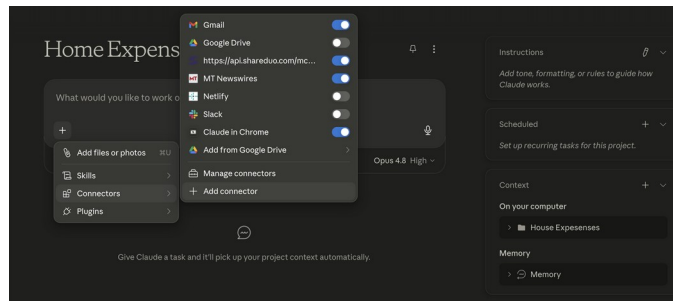
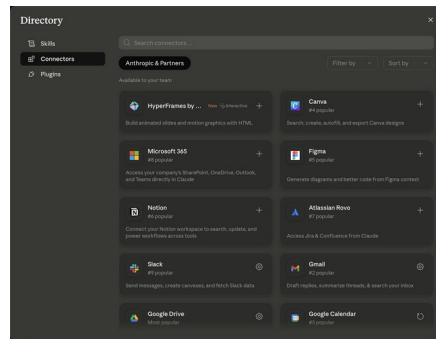
Connectors y MCP: las "manos" del agente

- **Qué son los Connectors.** Son lo que le permite al agente tocar sistemas externos que de otro modo no podría: Drive, Gmail, Slack, bases de datos, APIs. "Las manos: lo que el agente puede tocar que de otro modo no podría."
- **Qué es MCP (Model Context Protocol).** El estándar detrás de los Connectors: una forma estandarizada de conectar Claude con sistemas externos. Cualquier app que exponga un servidor MCP se vuelve algo con lo que podés "hablar" (Figma, Vercel, Cal.com, Home Assistant...). El patrón: la plataforma abre sus internals como herramientas MCP; el agente no gana una capacidad nueva, la plataforma se vuelve conversacional.
- En la próxima slide vemos **cómo se registra un Connector** en la práctica (directorio + un clic).



Cómo se registra un Connector

- **No estás programando: te conectás.** Registrar un Connector es como conectar Gmail a una app nueva — buscás el servicio en un directorio y le das **Connect**. Configurado por la UI; no hay archivo local que editar.
- **El directorio de Connectors.** Cowork trae un **directorío** con servicios listos para conectar de un clic:
- **Conexión de un clic.** Buscás el servicio, le das **Connect** y autorizás — y queda disponible para el agente:
- **Ejemplo (Atlas).** MT Newswires ya tiene un connector listo: lo buscás y le das Connect, como cualquier app. **Gmail**, igual: un clic en el directorio. Con eso, Atlas pasa a leer noticias de mercado y a dejar borradores de correo — sin que vos programes nada.



Schedule: que Cowork trabaje solo

- **Para qué sirve.** Todo lo anterior es reactivo (vos pedís, Claude hace). Schedule hace a Cowork **proactivo**: describís una tarea una vez, elegís una cadencia, y Claude la corre solo.
- **Cómo funciona.**
 - Describís la tarea una vez; Claude guarda el prompt como las instrucciones de la tarea.
 - Elegís cadencia: por hora · diaria · semanal · días de semana · o **a demanda** ("Run now").
 - Cada corrida abre su **propia sesión fresca de Cowork** y avisa al terminar.
 - Tiene los **mismos poderes** que una tarea normal: connectors, skills, plugins instalados.
 - Vive en la pestaña **Scheduled** de la barra lateral.
- ⚠ **LA trampa — corre LOCAL, no en la nube.** Las tareas programadas de Cowork corren **en tu computadora**, no en servidores de Anthropic. Solo se disparan **con la máquina encendida y la app de Claude Desktop abierta**. Si estaba dormida/cerrada a la hora prevista, la corrida se **saltea** — y se ejecuta automáticamente apenas la máquina despierta o reabrís la app (con aviso de "esto se saltó"). No esperes que tu laptop apagada genere el reporte del lunes.
- (Aparte, fuera de alcance:) existen agentes programados **alojados en la nube**, pero son una funcionalidad separada — no es lo que hace el Schedule de Cowork.
- **Ejemplo (Atlas).** Cada lunes 8:00: `buscar-accion` → `reporte-semanal` → dejar el reporte como borrador en Gmail, listo antes de la reunión de las 9:00. Tip de demo: no esperar al lunes, usar "Run on demand".

The screenshot shows a dark-themed dialog box titled "Create scheduled task" with a close button (X) in the top right corner. The dialog contains the following fields and options:

- Name ***: A text input field containing "daily-briefing".
- Description ***: A text input field containing "Summarize my calendar and inbox for the day".
- Instructions**: A larger text area containing the prompt "Check my Google Calendar for today's meetings and summarize my unread emails. Highlight anything urgent."
- Work in a project**: A dropdown menu showing a folder icon and the path "/Users/pveiga/Documents/Claude/Projects/House Expenses".
- Ask**: A toggle switch currently turned off.
- Default model**: A dropdown menu.
- Frequency**: A dropdown menu currently set to "Manual".
- Buttons**: "Cancel" and "Save" buttons at the bottom right.

05

Compartir resultados: Live Artifacts

Que la audiencia entienda qué es un Artifact, la distinción entre Artifact estándar y Live Artifact, y cómo se comparte el resultado con el equipo — cerrando el loop completo de la misión Atlas.

Artifacts y Live Artifacts: del resultado a algo compartible

- **Qué es un Artifact.** Una salida viva y ejecutable que se renderiza en un panel lateral: componentes React, páginas HTML, gráficos SVG, diagramas, tablas, documentos descargables.
- **Distinción live vs no-live (breve).**
 - **Artifact estándar** (todos los planes): salida de un solo archivo, estática — lo que generás es lo que queda.
 - **Live Artifact** (Cowork, planes pagos): una **página HTML interactiva y persistente** que vive en la pestaña **"Live artifacts"** de Cowork. **Se actualiza con datos actuales** de tus apps conectadas cada vez que la abris, y **guarda historial de versiones**.
- **Cómo se crea.** Dos formas: (1) **desde una tarea de Cowork** (le pedís que el resultado sea un Live Artifact), o (2) desde la pestaña **Live artifacts** → **New artifact** → **Chat with Claude**.
- **Estado actual del compartir — leer con cuidado.** Los Live Artifacts **todavía NO son compartibles**: en el lanzamiento son **para tu propio uso**; compartir está en el roadmap. Además son **locales, no en la nube**: viven en tu computadora y no te siguen entre dispositivos. Y **usan tus connectors sin volver a pedirte permiso** — solo los que aprobaste al crear/actualizar el artifact.
- **Ejemplo (Atlas).** El tablero `pulso-semanal-FECHA`: un Live Artifact nuevo por semana (queda un historial de versiones), con tarjetas por empresa, tabla resumen y un chip "LIVE", refrescado con los datos de la semana. Diseño basado en el boceto del jefe:



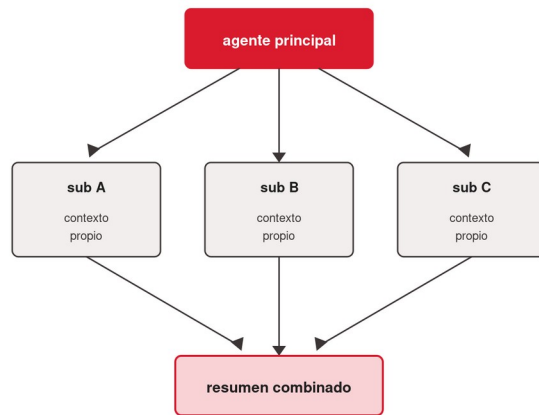
06

Advanced: Subagentes y Plugins

Dar un cierre de nivel avanzado: cómo Cowork delega trabajo pesado a Subagentes y cómo los Plugins empaquetan y distribuyen workflows completos (y son LA vía para meter una Skill en Cowork).

Subagentes: delegar sub-tareas en paralelo

- **Concepto.** Un Subagente es un asistente aislado, con su propio contexto, instrucciones y acceso a herramientas, al que el agente principal le delega un trabajo y del que recibe **un resumen** (no la transcripción completa).
- **Skill vs Subagente (la regla de una línea).** Chico, y debe quedar frente a vos → **Skill** (corre dentro de tu conversación). Grande o ruidoso, y debe correr en un proceso aparte → **Subagente** (corre al lado, en su propio contexto).
- **En Cowork.** Los Subagentes se coordinan "por debajo" (under the hood): el agente principal los lanza cuando le conviene, y pueden correr **varios en paralelo**.
- **Cómo se agrega un subagente.** Se define igual que una Skill — una **descripción de cuándo usarlo** + sus **instrucciones**. Dos caminos: **pedile a Claude que lo arme** (escribe el archivo del agente, como con las Skills, y lo gestionás en el directorio **Customize**), o viene **empaquetado dentro de un Plugin**. No hace falta tocar archivos a mano.

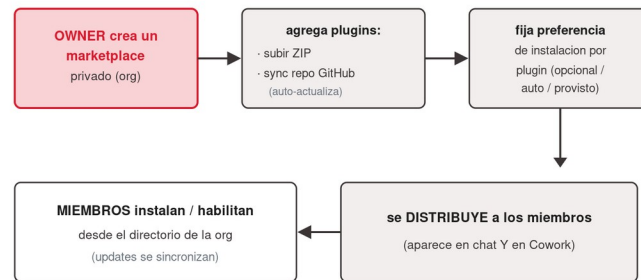


■ Plugins: empaquetar y distribuir un workflow completo

- **Concepto.** Un Plugin es la unidad de distribución de un workflow completo: empaqueta Skills + agentes + hooks + MCP en una sola instalación. "Ship the whole thing."
- **En Cowork.** Se instalan desde un **marketplace de plugins** en la GUI. Un Plugin es una de las vías para **distribuir Skills (y agentes/connectors)**: para usar una Skill en Cowork, la habilitas como skill de usuario (Customize → Skills) o la envías **dentro de un plugin**. Las skills provistas por plugin funcionan en Chat y en Cowork.
- **Dónde encontrarlos.** Marketplaces oficiales (`anthropics/claude-plugins-official`, `anthropics/knowledge-work-plugins`) y de la comunidad.

Plugins en una cuenta Team: ciclo de vida

- **Quién lo maneja.** En cuentas **Team / Enterprise**, los **Owners** gestionan los plugins de la organización desde **Organization settings** → **Plugins**. El resto de los miembros los reciben listos.
- **El ciclo de vida, de punta a punta:**
- **Qué se puede hacer.**
 - El Owner crea un **marketplace privado** de la organización y agrega plugins de dos formas: **subir un ZIP**, o **sincronizar desde un repo de GitHub** (privado) — esta segunda vía **auto-actualiza** cuando cambia el repo.
 - Por cada plugin se fija una **preferencia de instalación**: opcional (el miembro decide), **auto-install** o provisionado por usuario.
 - Los plugins distribuidos aparecen en **chat y en Cowork** para los miembros; cada uno **instala/habilita** desde el directorio de la org, y las **actualizaciones se sincronizan** solas.

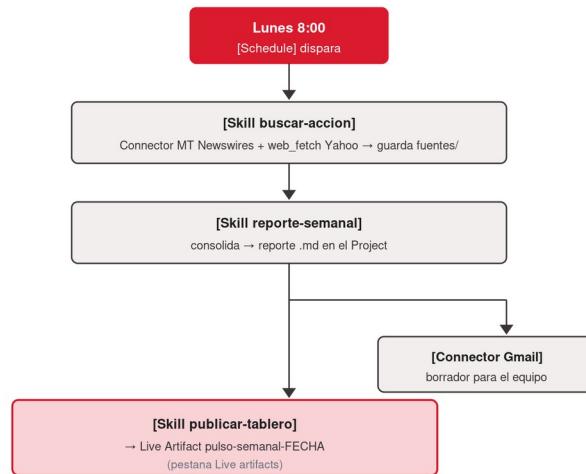




Conclusiones

El loop completo y la idea para llevarse

- Lo que construimos, punta a punta — el loop de Atlas combinando todas las piezas:
- **Las piezas, una línea cada una:** Instrucciones (el contrato) · Projects (el lugar fijo) · Skills (enseñar una vez) · Connectors/MCP (las manos) · Schedule (que corra solo) · Live Artifacts (compartir el resultado).
- **La idea para llevarse:** "Todo lo que le explicás a Claude dos veces es una Skill que deberías escribir una vez." Y el gancho: "Acaban de automatizar un reporte que les iba a comer la mañana de cada lunes. ¿Qué otra tarea recurrente podrían delegarle a su propio Atlas?"



Se delega una vez; corre solo cada lunes.

■ Gobernanza y advertencias (antes de Q&A)

- **Cowork no tiene audit trail** — no es apto para datos regulados o sensibles.
- **Toda salida es un borrador** — verificá cifras, citas y afirmaciones contra la fuente.
- **No metas datos confidenciales / PII / bajo NDA** en la superficie equivocada.
- **Reproducibilidad:** mantené juntos prompt + entradas + salidas, para que el trabajo sea auditable.
- **Capas de guardarraíles:** permisos de carpeta → reglas en Instrucciones → solo plugins verificados → revisión humana.
- Contexto biomédico: lo anterior es especialmente relevante si alguna vez aplican esto con datos de pacientes — Cowork no es la herramienta para datos clínicos sensibles.